

1 Theoretical Memory and Training-Time Model

This section develops a hardware-agnostic model for comparing the training requirements of BPTT, E-PROP, Forward-Forward (FF), and RATE-PEPITA. The analysis separates three quantities that capture complementary aspects of algorithmic cost:

1. **Memory footprint:** the number of scalar values that must be resident simultaneously during training.
2. **Compute cost:** the arithmetic workload generated by the forward dynamics and the learning rule.
3. **Memory-access cost:** the scalar read/write traffic generated while executing those operations.

The three quantities describe different resource demands. For example, E-PROP does not retain the complete temporal activation history, but its eligibility traces are updated at every time step. Its learning-specific storage can therefore remain independent of the temporal horizon while its compute and memory traffic still grow with the number of processed time steps.

The analysis assumes fully connected feedforward connectivity. Recurrent weight matrices and bias terms are excluded. Compute cost is expressed through dominant arithmetic operations implied by tensor dimensions, including synaptic accumulation, neuron-state updates, eligibility updates, feedback projection, spatial learning-signal propagation, and parameter updates. Memory-access cost is expressed as scalar read/write traffic for the corresponding quantities. These counts are intended to characterize algorithmic scaling; hardware-dependent execution effects are introduced only through the coefficients of the combined training-time proxy.

1.1 Common Notation

Let

$$B = \text{batch size}, \tag{1}$$

$$T = \text{number of time steps processed by one training instance}, \tag{2}$$

$$L = \text{number of hidden layers}, \tag{3}$$

$$d_0 = \text{original input dimension}, \tag{4}$$

$$d_l = \text{number of neurons in hidden layer } l, \tag{5}$$

$$C = \text{number of classes}. \tag{6}$$

Forward-Forward introduces class information by augmenting the input with C label channels. The effective input dimension to hidden layer l is therefore

$$\tilde{d}_{l-1} = \begin{cases} d_0 + C, & \text{FF and } l = 1, \\ d_0, & \text{non-FF methods and } l = 1, \\ d_{l-1}, & l > 1. \end{cases} \tag{7}$$

For hidden layer l , the number of feedforward synaptic weights is

$$a_l = \tilde{d}_{l-1} d_l. \tag{8}$$

The total number of hidden-layer synaptic weights is

$$A = \sum_{l=1}^L a_l, \quad (9)$$

and the total number of hidden neurons is

$$U = \sum_{l=1}^L d_l. \quad (10)$$

The quantity A is synapse-sized, whereas U is neuron-sized. Thus, A appears in terms associated with weights, eligibility traces, and weight updates, while U appears in terms associated with membrane states, spike activity, accumulated rates, and learning signals.

For methods with a separate classification readout, the number of readout weights is

$$V = Cd_L. \quad (11)$$

FF has no separate classification readout because the class label is incorporated at the input. Accordingly,

$$V = \begin{cases} 0, & \text{FF,} \\ Cd_L, & \text{BPTT, E-PROP, and RATE-PEPITA.} \end{cases} \quad (12)$$

With biases excluded, the total trainable parameter count is

$$N_{\text{param}} = A + V. \quad (13)$$

The input batch requires

$$N_{\text{input}} = BT\tilde{d}_0, \quad (14)$$

where $\tilde{d}_0 = d_0 + C$ for FF and $\tilde{d}_0 = d_0$ for the other methods.

1.2 Memory Decomposition

Peak training memory is decomposed as

$$N_{\text{training}} = N_{\text{input}} + N_{\text{param}} + N_{\text{state}} + N_{\text{intermediate}} + N_{\text{grad}}. \quad (15)$$

Here, N_{state} contains the neuron states required by temporal execution, $N_{\text{intermediate}}$ contains method-specific learning quantities, and N_{grad} contains parameter gradients or explicit weight-update matrices. The decomposition represents the peak resident working set under the execution schedule assumed for each method.

1.3 BPTT Memory

BPTT retains the hidden-neuron state across the full temporal horizon so that gradients can be propagated through the unrolled computation graph. The stored state is

$$N_{\text{state}}^{\text{BPTT}} = BTU. \quad (16)$$

Temporal intermediates required by the backward pass contribute another hidden-neuron-sized history together with output-side quantities:

$$N_{\text{intermediate}}^{\text{BPTT}} = BTU + BTC. \quad (17)$$

A gradient is retained for each trainable weight:

$$N_{\text{grad}}^{\text{BPTT}} = A + V. \quad (18)$$

Hence,

$$N_{\text{training}}^{\text{BPTT}} = BTd_0 + (A + V) + BTU + (BTU + BTC) + (A + V) \quad (19)$$

$$= BTd_0 + 2(A + V) + 2BTU + BTC. \quad (20)$$

The defining memory dependence is the temporal-history term BTU : neuron-sized information is retained for every processed time step.

1.4 E-PROP Memory

E-PROP maintains the current neuron state online rather than storing the complete temporal activation history:

$$N_{\text{state}}^{\text{E-PROP}} = BU. \quad (21)$$

An eligibility trace is maintained for each trainable synapse and each sample in the batch, requiring BA values. Layerwise learning signals require BU values, accumulated final-layer activity requires Bd_L , and the classifier-level output error requires BC . Therefore,

$$N_{\text{intermediate}}^{\text{E-PROP}} = BA + BU + Bd_L + BC. \quad (22)$$

Weight-update storage is

$$N_{\text{grad}}^{\text{E-PROP}} = A + V. \quad (23)$$

The resulting peak memory is

$$N_{\text{training}}^{\text{E-PROP}} = BTd_0 + (A + V) + BU + (BA + BU + Bd_L + BC) + (A + V) \quad (24)$$

$$= BTd_0 + 2(A + V) + BA + 2BU + Bd_L + BC. \quad (25)$$

The characteristic learning-memory term is BA , reflecting synapse-level eligibility storage rather than a time-unrolled neuron history.

1.5 Forward-Forward Memory

FF trains the network layer by layer. When layer k is active, preceding layers are frozen and provide the representation used to train the active layer. The corresponding state requirement is

$$N_{\text{state}}^{\text{FF}} = \max_{1 \leq k \leq L} \left[B \sum_{j=1}^{k-1} d_j + BTd_k \right]. \quad (26)$$

The local objective for the active layer requires its presynaptic representation and postsynaptic activity:

$$N_{\text{intermediate}}^{\text{FF}} = \max_{1 \leq k \leq L} BT(\tilde{d}_{k-1} + d_k). \quad (27)$$

Only the active layer is updated, so

$$N_{\text{grad}}^{\text{FF}} = \max_{1 \leq k \leq L} a_k, \quad (28)$$

and, because FF has no separate readout,

$$N_{\text{param}}^{\text{FF}} = A. \quad (29)$$

Thus,

$$N_{\text{training}}^{\text{FF}} = BT(d_0 + C) + A + \max_{1 \leq k \leq L} \left[B \sum_{j=1}^{k-1} d_j + BTd_k \right] \quad (30)$$

$$+ \max_{1 \leq k \leq L} BT(\tilde{d}_{k-1} + d_k) + \max_{1 \leq k \leq L} a_k. \quad (31)$$

Peak memory assumes that the positive and negative FF passes are scheduled sequentially. The dominant learning-specific storage is therefore determined by the active layer rather than by simultaneous storage across all layers.

1.6 RATE-PEPITA Memory

RATE-PEPITA uses two forward phases while retaining only the current neuron state and accumulated activity statistics. The state requirement is

$$N_{\text{state}}^{\text{RATE-PEPITA}} = BU. \quad (32)$$

Accumulated hidden-layer rates require $2BU$ values. In addition, RATE-PEPITA uses a fixed feedback matrix of size $C \times d_0$, an output error of size $B \times C$, and a projected input perturbation of size $B \times d_0$. Hence,

$$N_{\text{intermediate}}^{\text{RATE-PEPITA}} = 2BU + Cd_0 + BC + Bd_0. \quad (33)$$

The peak-memory schedule retains two neuron-sized rate buffers, one for each phase, and applies the projected perturbation in place, avoiding an additional sequence-sized BTd_0 buffer. Weight-update storage is

$$N_{\text{grad}}^{\text{RATE-PEPITA}} = A + V. \quad (34)$$

Therefore,

$$N_{\text{training}}^{\text{RATE-PEPITA}} = BTd_0 + (A + V) + BU + (2BU + Cd_0 + BC + Bd_0) + (A + V) \quad (35)$$

$$= BTd_0 + 2(A + V) + 3BU + Cd_0 + BC + Bd_0. \quad (36)$$

RATE-PEPITA replaces temporal-history and synapse-level learning storage with neuron-sized accumulated-rate buffers.

1.7 Compute Model

The compute model estimates the arithmetic workload of one training step from the dominant operations performed by each learning rule. The counts scale directly with the dimensions of the corresponding matrix-vector, state-update, trace-update, and parameter-update operations.

1.7.1 BPTT Compute

A BPTT training step contains one temporal forward phase and one temporal backward phase. Each traverses the network over B samples and T time steps with dominant workload $BT(A + V + U)$. Including output-side temporal processing gives

$$N_{\text{compute}}^{\text{BPTT}} = 2BT(A + V + U) + BTC. \quad (37)$$

1.7.2 E-PROP Compute

E-PROP performs temporal network execution with cost $BT(A + V + U)$ and updates synaptic eligibility traces throughout the sequence with cost BTA . Spatial propagation of the learning signal is represented by one network-wide traversal $B(A + V)$, and the final parameter update contributes another $B(A + V)$. Including output-error processing gives

$$N_{\text{compute}}^{\text{E-PROP}} = BT(A + V + U) + BTA + 2B(A + V) + BC. \quad (38)$$

1.7.3 Forward-Forward Compute

During a complete layerwise sweep, layer l is executed $L - l + 1$ times because it is revisited whenever a deeper layer is trained. Each execution contributes $a_l + d_l$, and positive and negative examples introduce a factor of two. The repeated forward executions therefore contribute

$$2BT \sum_{l=1}^L (L - l + 1)(a_l + d_l).$$

The local positive/negative update of each active layer contributes one additional traversal,

$$2BT \sum_{l=1}^L (a_l + d_l).$$

Combining the two components gives

$$N_{\text{compute}}^{\text{FF}} = 2BT \sum_{l=1}^L (L - l + 2)(a_l + d_l). \quad (39)$$

1.7.4 RATE-PEPITA Compute

RATE-PEPITA performs two complete temporal forward phases, contributing $2BT(A + V + U)$. The fixed feedback projection requires BCd_0 , output-error processing contributes BC , and accumulated-rate parameter updates contribute $B(A + V)$. Thus,

$$N_{\text{compute}}^{\text{RATE-PEPITA}} = 2BT(A + V + U) + B(Cd_0 + C) + B(A + V). \quad (40)$$

1.8 Memory-Access Model

The memory-access model estimates the scalar traffic generated by the same training procedure. It accounts for weight reads, activity reads, neuron-state updates, learning-state updates, and parameter reads/writes. The purpose of this model is to distinguish algorithms that have similar arithmetic workload but different data-movement requirements.

1.8.1 Forward Traffic

For hidden layer l , define

$$f_l = a_l + \tilde{d}_{l-1} + 2d_l, \quad (41)$$

where a_l represents weight access, $\tilde{d}_{l-1}$ represents input-activity access, and $2d_l$ represents a read and write of neuron state/activity. The total hidden-layer forward traffic is

$$F = \sum_{l=1}^L f_l. \quad (42)$$

1.8.2 Update Traffic

For hidden layer l , define

$$p_l = a_l + \tilde{d}_{l-1} + d_l, \quad (43)$$

where the three terms represent weight/update access, presynaptic-activity access, and postsynaptic activity, rate, or learning-signal access. The total hidden-layer update traffic is

$$P = \sum_{l=1}^L p_l. \quad (44)$$

1.8.3 Readout Traffic

For methods with a classification readout, define

$$Y = V + d_L + C. \quad (45)$$

For FF, which has no separate readout,

$$Y = 0. \quad (46)$$

The forward term F accounts for ordinary network execution. Additional method-specific terms below account for data movement associated with retained temporal quantities, eligibility traces, accumulated rates, spatial learning signals, and parameter updates.

1.8.4 BPTT Memory Access

BPTT memory traffic is modeled as

$$N_{\text{access}}^{\text{BPTT}} = BT(F + Y) + 4BTU + 2BTC + BT(P + Y) + 2(A + V). \quad (47)$$

The terms correspond to temporal forward execution, read/write traffic for retained hidden states and intermediates, temporal output quantities, backward/update traffic, and final parameter read/write traffic.

1.8.5 E-PROP Memory Access

E-PROP performs ordinary temporal forward execution, reads and writes each eligibility trace at every time step, propagates the spatial learning signal, and applies the final parameter update. The resulting access model is

$$N_{\text{access}}^{\text{E-PROP}} = BT(F + Y) + 2BTA + 2B(P + Y) + 2(A + V). \quad (48)$$

The factor $2BTA$ accounts for one read and one write of each eligibility trace during its online update. The two $B(P + Y)$ terms represent the spatial learning-signal traversal and the final update traversal.

1.8.6 Forward-Forward Memory Access

FF memory traffic is

$$N_{\text{access}}^{\text{FF}} = 2BT \sum_{l=1}^L (L - l + 1) f_l + 2BTU + 2BT \sum_{l=1}^L p_l + 2A. \quad (49)$$

These terms represent repeated positive/negative forward execution, active-layer neuron-state traffic, local layer-update traffic, and hidden-weight read/write traffic.

1.8.7 RATE-PEPITA Memory Access

RATE-PEPITA memory traffic is

$$N_{\text{access}}^{\text{RATE-PEPITA}} = 2BT(F + Y) + 4BTU + B(Cd_0 + C + d_0) + B(P + Y) + 2(A + V). \quad (50)$$

The feedback-related contribution is

$$B(Cd_0 + C + d_0) = BCd_0 + BC + Bd_0, \quad (51)$$

covering the feedback projection, output error, and projected input perturbation. The $4BTU$ term accounts for rate/state data movement across the two forward phases in addition to the ordinary forward traffic represented by $2BT(F + Y)$.

1.9 Combined Training-Time Proxy

Arithmetic workload and memory traffic contribute differently to execution time depending on the target platform. For method m , the combined proxy is

$$N_{\text{time}}^{(m)} = \alpha N_{\text{compute}}^{(m)} + \beta N_{\text{access}}^{(m)}, \quad (52)$$

where

$$\alpha = \text{effective time cost per unit of arithmetic workload}, \quad (53)$$

$$\beta = \text{effective time cost per scalar memory access}. \quad (54)$$

The coefficients remain symbolic so that the same algorithmic model can represent different compute-to-memory cost ratios across hardware platforms and memory hierarchies.

1.10 Characteristic Scaling of the Learning Rules

The detailed expressions above reduce to distinct dominant storage behaviors:

Method	Main stored learning quantity	Characteristic scaling
BPTT	Temporal neuron history	BTU
E-PROP	Synaptic eligibility traces	BA
FF	Active-layer quantities	layerwise maximum
RATE-PEPITA	Accumulated rates	BU

These terms summarize the principal storage mechanism of each learning rule: BPTT scales with temporal history, E-PROP with synapse-level traces, FF with the active-layer working set, and RATE-PEPITA with neuron-sized accumulated rates.

1.11 Effect of Temporal Windowing

Let T_{full} denote the original sequence length. A window has length S , and the hop ratio is H . The corresponding hop length is

$$q = HS, \quad (55)$$

with q taken as an integer at the level of the analysis. When only complete windows are retained, the number of windows extracted from one original sequence is

$$N_w = \left\lfloor \frac{T_{\text{full}} - S}{q} \right\rfloor + 1, \quad S \leq T_{\text{full}}. \quad (56)$$

For peak memory, windows are processed sequentially. The temporal dimension in the per-window memory equations is therefore

$$T = S, \quad (57)$$

and the peak training-memory requirement for method m becomes

$$N_{\text{training,window}}^{(m)} = N_{\text{training}}^{(m)}|_{T=S}. \quad (58)$$

Thus, terms proportional to T scale with the selected window length, whereas non-temporal terms such as A , V , BA , and Cd_0 are unchanged. For example, the BPTT temporal state term changes from $BT_{\text{full}}U$ to BSU .

Total workload must also account for the number of windows processed. For a batch of sequences that each produce N_w windows,

$$N_{\text{compute,total}}^{(m)} = N_w N_{\text{compute}}^{(m)}|_{T=S}, \quad (59)$$

$$N_{\text{access,total}}^{(m)} = N_w N_{\text{access}}^{(m)}|_{T=S}. \quad (60)$$

The corresponding training-time proxy is

$$N_{\text{time,total}}^{(m)} = N_w \left[\alpha N_{\text{compute}}^{(m)}|_{T=S} + \beta N_{\text{access}}^{(m)}|_{T=S} \right]. \quad (61)$$

The total temporal workload per original sequence is

$$T_{\text{processed}} = N_w S. \quad (62)$$

For overlapping windows ($H < 1$), $T_{\text{processed}}$ can exceed T_{full} : the per-window temporal footprint decreases with S , while total compute and memory traffic may increase because more windows are evaluated. With non-overlapping windows ($H = 1$), the processed temporal workload approaches the original sequence length up to discarded boundary samples. For $H > 1$, gaps reduce temporal coverage and can lower total workload.

Windowing therefore affects the model through two separate quantities: S determines the temporal horizon and peak per-window storage, while N_w determines how many per-window compute and access costs contribute to the complete workload.

1.12 Summary

The model separates peak storage, arithmetic workload, and memory traffic while retaining the distinct execution structure of each learning rule. The characteristic storage behavior is summarized by

BPTT: BTU	E-PROP: BA	FF: active-layer maximum	RATE-PEPITA: BU
-------------	--------------	--------------------------	-------------------

and temporal windowing introduces a second level of scaling: S controls the cost of an individual window, whereas N_w controls the number of windows processed over the original sequence.